SOFTWARE REQUIREMENTS SPECIFICATION

Hospital Management System

(C++ / Qt Desktop Application)

Supervised By:

Dr. Lamiaa

Eng. Abdelrahman Gaber

Eng. Reem Waleed

Eng. Asmaa El Baghdady

Prepared By:

Mai Ahmed Khalaf	192400685	[G.3-2]
Zeyad Waleed Amin	192400694	[G.3-2]
Adam Tamer Ghobashy	192400667	[G.3-2]
Judy Ehab Abdelmagied	192400739	[G.3-3]
Basma Mohamed Ibrahim	192400703	[G.3-2]
Hazem Mohamed Hmady	192400671	[G.3-2]
Maha Mohamed Nasr	192400778	[G.3-3]

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the complete functional and non-functional requirements for the Hospital Management System (HMS), a C++ desktop application designed to streamline and automate key hospital operations. Version 1.0 covers the core modules: Patient Registration, Doctor Management, Appointment Scheduling, Billing, and basic Reporting.

This document serves as the authoritative agreement between stakeholders and the development team, and as the baseline for system design, development, and testing.

1.2 Document Conventions

The following conventions are used throughout this document:

- REQ-XXX: Unique requirement identifiers where XXX is a sequential number.
- Priority levels: High (must-have), Medium (should-have), Low (nice-to-have).
- C++ references assume ISO C++17 standard unless otherwise stated.
- Bold text highlights key terms or important notes.
- All data types reference standard C++ types (std::string, int, double, etc.).

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the following audiences:

- Developers: Focus on Sections 3, 4, and 5 for implementation details.
- Project Managers: Focus on Section 4 for feature scope and Section 5 for quality attributes.
- Testers: Focus on Section 4 for functional requirements and Section 5.1 for performance targets.
- End Users (Hospital Staff): Focus on Section 2.3 for user roles and Section 4 for system features.

It is recommended to read Sections 1 and 2 first for context, then proceed to the sections most relevant to your role.

1.4 Product Scope

The Hospital Management System (HMS) is a standalone C++ desktop application intended for small to medium-sized hospitals and clinics. The system digitizes and centralizes hospital workflows that are currently managed manually or across disconnected tools.

Core objectives of the HMS include:

- Reducing patient registration and check-in time by 60% compared to manual processes.
- Providing instant access to patient medical records and appointment history.
- Automating billing calculations including consultation fees and medicine charges.
- Enabling hospital administrators to generate operational reports on demand.
- Eliminating scheduling conflicts through automated appointment management.

The system is not intended to replace specialized clinical software (e.g., PACS for medical imaging) and does not include telemedicine or insurance claim processing in this version.

1.5 References

- ISO/IEC 12207 — Software Lifecycle Processes
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- ISO C++17 Standard (ISO/IEC 14882:2017)
- HL7 FHIR R4 — Health Level 7 Fast Healthcare Interoperability Resources (included for future interoperability consideration only)

2. Overall Description

2.1 Product Perspective

The HMS is a new, self-contained desktop application being developed from scratch in C++. It operates as a standalone system that does not depend on external cloud services or APIs in its initial version. Data is persisted locally using structured text files stored on the system.

The system interfaces directly with hospital staff (admins, doctors, receptionists) through a graphical user interface built using Qt. It replaces existing paper-based and spreadsheet workflows for patient registration, appointment tracking, and billing.

2.2 Product Functions

At a high level, the HMS provides the following major functional areas:

- Patient Management
- Doctor Management
- Appointment Scheduling
- Medical Records
- Billing & Invoicing
- Reporting & Analytics

2.3 User Classes and Characteristics

The HMS supports two primary user classes:

User Class	Technical Level	Description
Doctor	Basic	Views patient records and medical history only.
Receptionist	Basic to Intermediate	Books appointments, checks out patients, views patient data, handles billing.

2.4 Operating Environment

- Operating System: Windows 10/11 (primary), Linux Ubuntu 20.04+ (secondary)
- Compiler: GCC 9+ or MSVC 2019+ with C++17 support
- Memory: Minimum 512 MB RAM; 2 GB recommended
- Storage: Minimum 200 MB for application; 1 GB for data files
- Display: 1024x768 minimum resolution (Qt-based graphical desktop UI in v1.0)
- Database: Local structured text files for data storage

2.5 Design and Implementation Constraints

- The system must be implemented entirely in standard ISO C++17 with no proprietary extensions.
- Qt framework is used for building the graphical user interface.
- The C++ Standard Library (STL) is used for core programming features and file handling.
- Data persistence uses structured text files for storing system records.
- The application must compile and run on both Windows and Linux without code changes.
- Patient data handling must comply with data protection principles (no plain-text passwords).
- The system is single-user per session; concurrent multi-user access is not required in v1.0.

2.6 User Documentation

- User Manual (PDF): Step-by-step guide for all user roles covering all system features.
- Quick Reference Card: One-page laminated card for receptionists covering daily workflows.
- In-app Help: Context-sensitive help text accessible from any menu screen.
- README.md: Developer setup guide including build instructions and dependencies.

2.7 Assumptions and Dependencies

- It is assumed that hospital staff can read and write in English.
- It is assumed each workstation has a single local installation of the HMS.
- Patient IDs are manually assigned by the admin at the time of registration.
- The project depends on local text files for saving and retrieving data.
- The operating system provides normal file access permissions.

3. External Interface Requirements

3.1 User Interfaces

The following screenshots illustrate the Al-Nour Hospital System graphical user interface as implemented in the current version. The graphical user interface is implemented using the Qt Widgets framework with multi-window navigation for core hospital workflows.

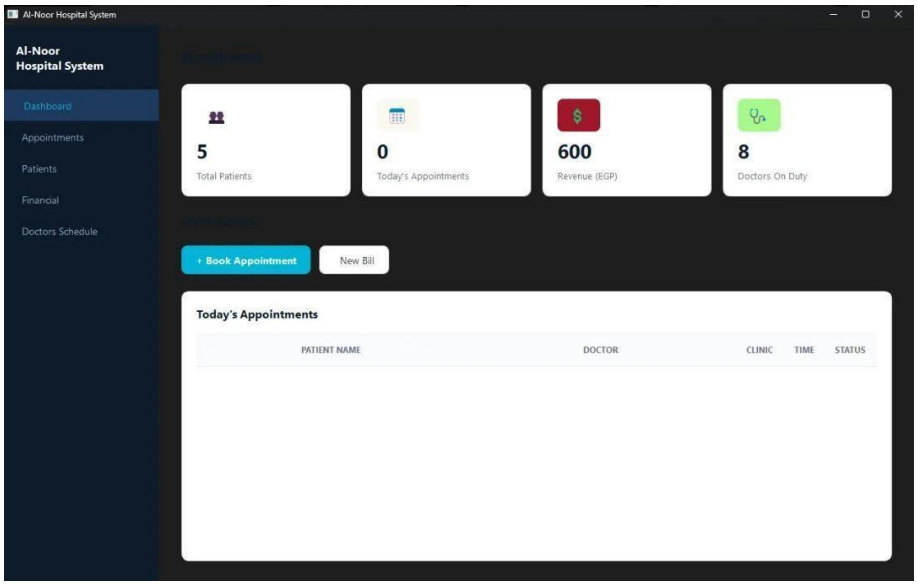


Figure 1: Main Dashboard — displays real-time summary of total patients, today's appointments, revenue, and doctors on duty

Figure 2: Patient Management — patient list panel with search and add new patient functionality

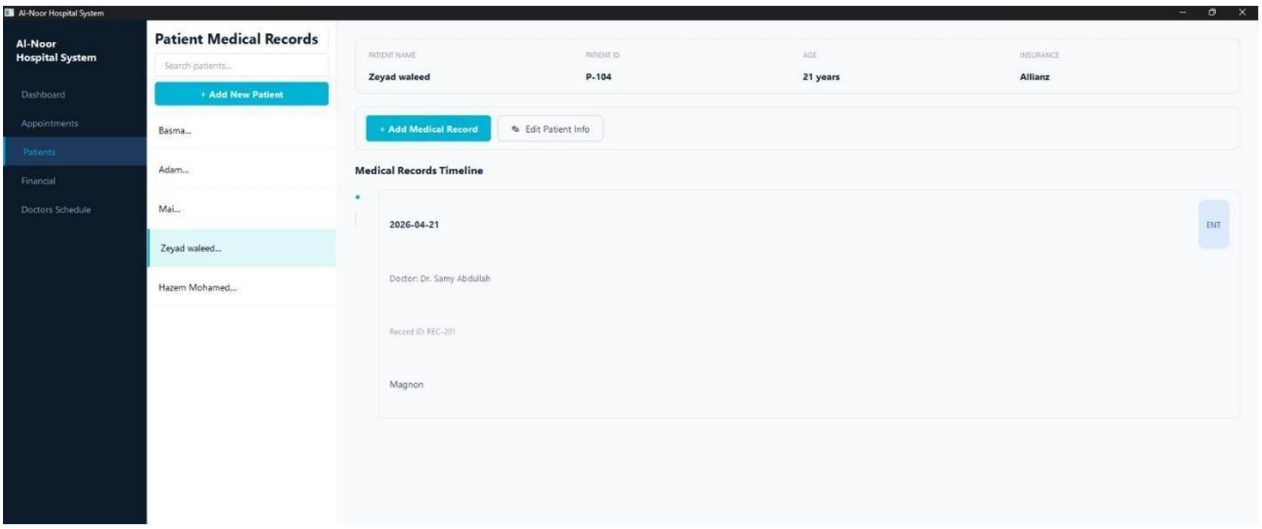


Figure 3: Patient Medical Records — selected patient details and medical records timeline

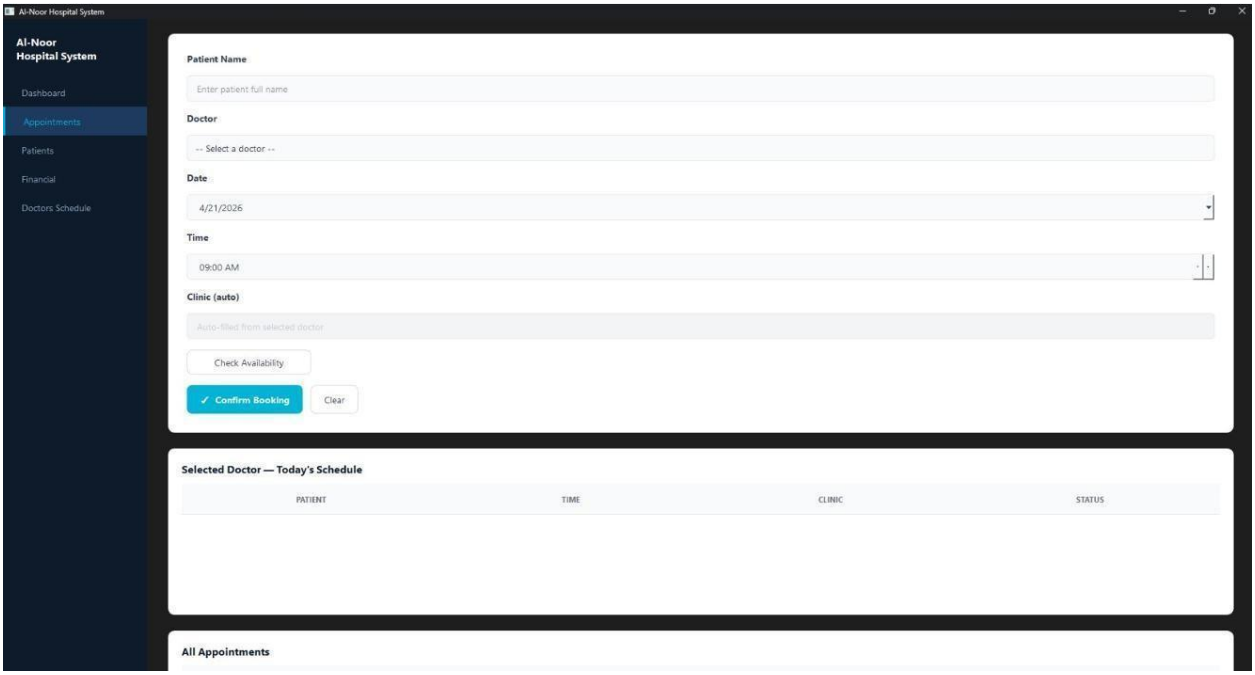


Figure 4: Appointment Booking — form for scheduling appointments with availability checking and today's doctor schedule

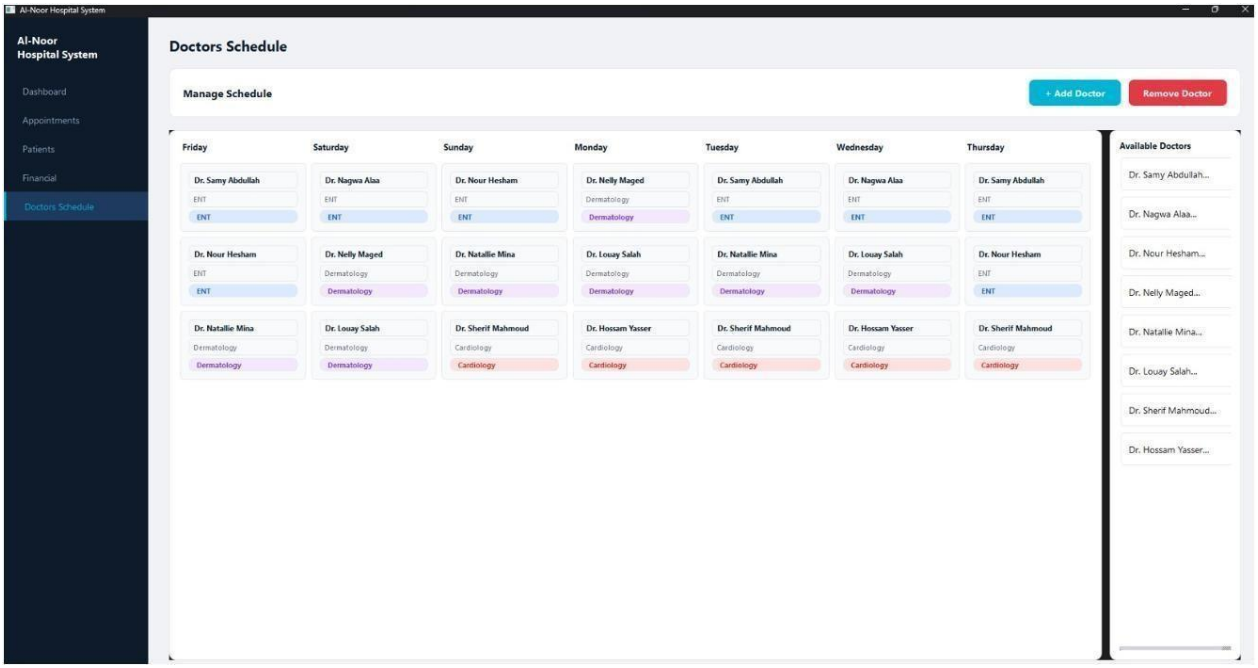
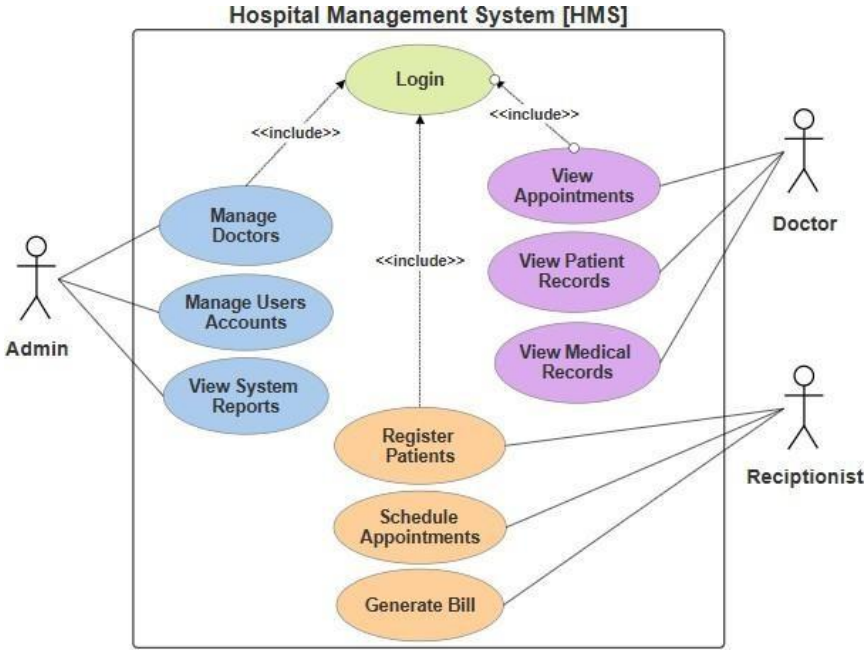


Figure 5: Doctors Schedule — weekly schedule calendar with specialization-coded doctor assignments

Link:

https://www.figma.com/make/OaFT4mH0KySLfhhEsafaQL/Untitled?t=U76pxIBsgvBBZMpF-1&preview-route=%2Fpatients

3.2 Use-Case Diagram



4. System Features

4.1 Patient Management

4.1.1 Description and Priority

Allows hospital staff to create and update patient records. This is the foundational module of the HMS and is of HIGH priority. All other modules depend on valid patient records existing in the system.

4.1.2 Stimulus/Response Sequences

- User selects 'Patient Management' from the main menu → System displays patient sub-menu.
- User selects 'Register New Patient' → System prompts for patient details → validates input → saves the manually entered Patient ID → saves record → confirms success with ID.
- User selects 'Search Patient' → System prompts for name or ID → displays matching records in table format.
- User selects 'Update Patient' → System prompts for Patient ID → displays current data → allows field-by-field update → saves changes.
- User selects 'View Patient History' → System displays all appointments, diagnoses, and bills linked to that patient.

4.1.3 Functional Requirements

Req. ID	Description	Priority
REQ-001	The system shall store: full name, age, patient ID (manually assigned), and insurance company (optional, one of 3 contracted companies).	**High**
REQ-002	The system shall validate that the manually entered ID is unique; duplicate IDs shall be rejected with an error message.	**High**
REQ-003	The system shall support searching patients by name (partial match), Patient ID, or ID.	**High**
REQ-004	The system shall display a complete patient history including all visits and diagnoses.	**Medium**

4.2 Doctor Management

4.2.1 Description and Priority

Manages doctor profiles, specializations, and availability schedules. Priority: HIGH. The appointment system cannot function without accurate doctor records.

4.2.2 Stimulus/Response Sequences

- Admin selects 'Add Doctor' → System prompts for doctor details → validates → assigns Doctor ID → saves.
- Admin selects 'Set Availability' → System displays doctor's weekly schedule → admin sets available days/hours → system saves schedule.
- Any user selects 'List Doctors by Specialization' → System prompts for specialty → displays all available doctors.

4.2.3 Functional Requirements

Req. ID	Description	Priority
REQ-005	The system shall store doctor profiles: name, specialization/clinic, and weekly schedule (available days and hours).	High
REQ-006	The system shall allow admins to define a doctor's available time slots per day of the week.	High
REQ-007	The system shall support filtering doctors by specialization.	Medium
REQ-008	The system shall display a doctor's current appointment load (number of appointments today), availability is defined as time ranges (from hour X to hour Y) per day.	Low
REQ-009	The system shall allow administrators to permanently remove a doctor's profile from the system, including their schedule and associated data.	High

4.3 Appointment Scheduling

4.3.1 Description and Priority

Core scheduling feature for booking, modifying, and canceling appointments between patients and doctors. Priority: HIGH. This is the most frequently used feature by receptionists.

4.3.2 Stimulus/Response Sequences

- Receptionist selects 'Book Appointment' → Enters Patient ID and preferred Doctor ID → System checks availability → Shows available time slots → Receptionist selects slot → System confirms booking and generates Appointment ID.
- User selects 'Cancel Appointment' → Enters Appointment ID → System prompts for confirmation → Marks as canceled → Updates doctor's availability.
- Doctor logs in → Selects 'My Appointments Today' → System displays sorted list of today's appointments with patient names and times.

4.3.3 Functional Requirements

Req. ID	Description	Priority
REQ-010	The system shall assign a unique Appointment ID to each booked appointment processed through the queue.	High
REQ-011	The system shall prevent double-booking: a doctor may not have two appointments at the same date and time.	High
REQ-012	The system shall decline any appointment booking request if the selected doctor is not available during the requested time.	High
REQ-013	Appointment status shall be tracked as one of: Scheduled, Completed, or Canceled.	High
REQ-014	The system shall display all appointments for a given date, filterable by doctor or department.	Medium

4.4 Medical Records

4.4.1 Description and Priority

Manages patient visit records, and diagnoses. Priority: HIGH for record creation; MEDIUM for advanced querying.

4.4.2 Stimulus/Response Sequences

- Doctor selects a completed appointment → System displays patient's record form → Doctor enters diagnosis → System saves the visit record.
- User searches a patient → Views 'Medical History' → System displays chronological list of all visit records.

4.4.3 Functional Requirements

Req. ID	Description	Priority
REQ-015	The system shall create a medical record linked to each completed appointment.	High
REQ-016	Each medical record shall store: date, doctor ID, and diagnosis.	High
REQ-017	The system shall display a patient's full medical history in reverse chronological order.	High

4.5 Billing & Invoicing

4.5.1 Description and Priority

Generates and manages patient bills covering consultation fees, medicines, and procedures. Priority: HIGH — directly impacts hospital revenue.

4.5.2 Stimulus/Response Sequences

- Receptionist selects 'Financial' from the main menu → System displays the billing screen.
- Receptionist enters Patient ID → System retrieves the patient's information.
- Receptionist selects the clinic → System automatically displays the base price for that clinic.
- System checks if the patient has insurance → If yes, displays the insurance provider and the discount amount → Calculates and displays the final amount the patient pays.
- Receptionist enters payment details (card holder name, card number, expiry date, CVV) → System masks the card number and CVV for security → Receptionist clicks 'Process Payment' → System confirms the payment.
- System saves the bill with a unique Bill ID and records it in the Recent Billing Records table showing Bill ID, Patient ID, service, amount, and date.

4.5.3 Functional Requirements

Req. ID	Description	Priority
REQ-018	The system shall generate a unique Bill ID for each invoice (format: BILL-YYYYMMDD-XXXX).	High
REQ-019	Bills shall be linked to a specific Appointment ID and Patient ID.	High
REQ-020	Each bill shall consist of a fixed consultation fee determined by the patient's clinic, with an insurance discount applied if the patient has insurance.	High
REQ-021	The system shall automatically calculate the patient's payable amount by deducting the contracted insurance company's specific discount from the full clinic price.	High
REQ-022	The system shall support credit card as the only accepted payment method.	High
REQ-023	The system shall mask the credit card CCV during entry to ensure secure payment processing.	High
REQ-024	The system shall not store any credit card data after the transaction is completed; card details are used solely for payment processing.	High
REQ-025	The system shall have a fixed consultation fee configured for each clinic.	High
REQ-026	The system shall support 3 contracted insurance companies, each with a specific discount amount that is deducted from the full clinic price at checkout.	High
REQ-027	The system shall display a billing records table in the Financial section showing all recent bills with Bill ID, Patient ID, service, amount, and date.	Medium

4.6 Reporting & Analytics

4.6.1 Description and Priority

Provides hospital staff with a real-time financial summary and billing history to support operational oversight. Priority: MEDIUM. This module presents aggregated financial data derived from processed bills.

4.6.2 Stimulus/Response Sequences

- User navigates to 'Financial' from the main menu → System displays real-time summary cards showing total patient payments, total insurance coverage, and net hospital revenue.
- User views the Recent Billing Records table → System displays a chronological log of all processed bills including Bill ID, Patient ID, service, amount, and date.

4.6.3 Functional Requirements

Req. ID	Description	Priority
REQ-028	The system shall display the total amount paid by patients across all processed bills.	**Medium**
REQ-029	The system shall display the total amount covered by insurance across all processed bills.	**Medium**
REQ-030	The system shall display the net hospital revenue as the sum of patient payments and insurance coverage.	**Medium**
REQ-031	The system shall maintain and display a recent billing records table showing Bill ID, Patient ID, service, amount, and date for all processed transactions.	**Medium**

4.7 User Authentication & Access Control

4.7.1 Description and Priority

The system operates under a single admin account with access to all modules. There are no individual user accounts for doctors or receptionists — the system is accessed through one shared login. Certain sensitive areas such as financial data editing are restricted to prevent unauthorized modification and ensure data integrity.

4.7.2 Stimulus/Response Sequences

• User launches the system → System displays login screen → User enters the admin username and password → if valid, full access to all modules is granted except restricted areas such as financial data editing.

• User enters invalid credentials → System displays an error message and prompts to retry.

4.7.3 Functional Requirements

Req. ID	Description	Priority
REQ-032	The system shall require username and password authentication before any access is granted.	High
REQ-033	Passwords shall be stored as SHA-256 hashes; plain-text passwords shall never be stored.	High
REQ-034	The system shall operate under a single admin account with access to all modules.	High
REQ-035	The system shall restrict editing of financial data to protect system integrity.	High

5. Other Non-Functional Requirements

5.1 Performance Requirements

- Patient search by ID or name shall return results within 1 second for a data storage of up to 10,000 patient records.
- Appointment booking confirmation shall complete within 2 seconds.
- Bill generation shall complete within 3 seconds including tax computation.
- The system shall support a data store of at least 10,000 patient records, 500 doctors, and 100,000 appointment records while maintaining acceptable performance for small-to-medium hospital deployments.
- Application start-up (cold start to login screen) shall not exceed 3 seconds on the minimum hardware configuration.

5.2 Safety Requirements

- The system shall perform an automatic data backup to a secondary file every 24 hours.
- If the application crashes during a write operation, data integrity shall be preserved via transactional writes (using safe file write operations and backup copies).
- The system shall not expose internal C++ stack traces or file paths in user-facing error messages.

5.3 Security Requirements

- All passwords must be hashed using SHA-256; no password recovery mechanism shall reveal the original password.
- All data files shall be stored in a protected system directory with appropriate file access permissions. Sensitive credentials such as passwords shall be securely hashed, and administrative access to raw data files shall be restricted.
- The system shall comply with the principle of least privilege: each user role has access only to the functions required for their job.
- The system shall mask the credit card CCV during entry and shall not store any card data after the transaction is completed.
- Editing of financial data shall be restricted to protect system integrity.

5.4 Software Quality Attributes

- Usability: A new receptionist with basic computer literacy shall be able to complete patient registration within 3 minutes after 2 hours of training.
- Reliability: The system shall have a Mean Time Between Failures (MTBF) of at least 200 hours of continuous operation.
- Maintainability: All C++ classes shall be documented with Doxygen-style comments. Cyclomatic complexity per function shall not exceed 15.
- Portability: The codebase shall compile without modification on both Windows (MSVC 2019+) and Linux (GCC 9+) using a provided CMakeLists.txt.

5.5 Business Rules

- Patients cannot be removed from the system once registered.
- Each clinic has a fixed consultation fee.
- Insurance discounts are applied at checkout based on the patient's contracted insurance company.

6. Other Requirements

The following additional requirements apply to the HMS:

- Data Storage Structure: Separate structured text files (.txt/.csv) shall be used for patients, doctors, appointments, medical records, bills, users, and logs.
- Internationalization: In v1.0, the system supports English only. String literals shall be isolated in a single constants header to facilitate future localization.
- Configuration: A plain-text config file (config.ini) shall store settings such as tax rate, backup interval, session timeout, and database path. The application shall load this file on startup.
- Build System: A CMakeLists.txt shall be provided supporting both Debug and Release build configurations. The Release build shall enable compiler optimizations (-O2).
- Coding Standards: Code shall follow the Google C++ Style Guide with the exception of using 4-space indentation instead of 2-space.

Appendix A: Glossary

Term	Definition
HMS	Hospital Management System — the software product described in this SRS.
Patient ID	A manually assigned unique identifier for each registered patient.
Doctor ID	A unique identifier for each doctor in the system.
Specialization	A doctor's field of medical expertise (e.g., Cardiology, Pediatrics, General Practice).
STL	C++ Standard Template Library — the standard collection of C++ data structures and algorithms.
Text File Storage	A simple local storage method using structured text files to save data records.
SHA-256	A cryptographic hash function used to securely store passwords in the system.
MTBF	Mean Time Between Failures — a measure of system reliability.
SRS	Software Requirements Specification — this document.

Appendix B: Analysis Models

Class Architecture Overview

The HMS follows an object-oriented architecture. The core C++ classes and their relationships are described below:

- Person (base class): Attributes shared by Patient and Doctor — name..
- Patient (extends Person): patientId (manually assigned), age, insuranceCompany (optional)..
- Doctor (extends Person): doctorId, name, specialization, schedule (map of available days and working hours).
- Appointment: appointmentId, patientId, doctorId, dateTime, status (enum: SCHEDULED/COMPLETED/CANCELED).
- MedicalRecord: recordId, patientId, doctorId, clinic, diagnosis.
- Bill: patientId, clinicFee, insuranceDiscount, amountPaid, paymentMethod (credit card only).
- HospitalSystem: Central controller class managing all modules, authentication state, and data access under a single admin account.
-

Appendix C: To Be Determined (TBD) List

TBD #	Description	Target Resolution
TBD-01	Exact tax rate percentage to be configured in the billing module.	Before development of Billing module
TBD-02	Final list of medical specializations to be supported in doctor profiles.	Requirements review meeting
TBD-03	Which hashing library will be used for password hashing.	Architecture decision — Sprint 1